

HERMES STACK

Blueprint Técnico Definitivo

Arquitectura · Pipeline de Voz · CI/CD · Observabilidad

Autor: Erik José Hernández
Email: erikjosehernandez@gmail.com
GitHub: github.com/erikjosehrndz/hermes-stack
Dominio: el80.space
Versión: 2.0 — Mayo 2026

Plataforma de IA conversacional autohospedada en VPS único.
10 microservicios Docker · HTTPS automático · Pipeline de voz <500 ms

Índice general

1. Arquitectura General del Sistema	1
1.1. Resumen Ejecutivo	1
1.1.1. Métricas objetivo	1
1.2. Diagrama General del Sistema	1
1.3. Principios Arquitectónicos	2
1.4. Topología de Red Docker	2
1.4.1. Puertos expuestos al host	3
1.5. Hardening de Seguridad en Contenedores	3
2. Pipeline de Voz e Inventario de Servicios	5
2.1. Pipeline End-to-End de Voz	5
2.1.1. Latencias por etapa	5
2.1.2. Optimizaciones de latencia	6
2.1.3. Política de fallback en LiteLLM	6
2.2. Inventario Detallado de Servicios	6
2.2.1. LiteLLM Router (:4000)	6
2.2.2. Hermes Agent (:8080)	7
2.2.3. Whisper STT (:9000)	7
2.2.4. Redis Cache	8
2.2.5. Hermes Website (:3001)	8
2.2.6. File Browser (:8095)	8
3. CI/CD, Despliegue y App Android	9
3.1. Pipeline CI/CD — GitHub Actions	9
3.1.1. Etapa 1 — Lint & Validation	9
3.1.2. Etapa 2 — Despliegue al VPS	10
3.1.3. Secretos requeridos en GitHub	10
3.1.4. Watchdog del Docker Daemon	10
3.2. Configuración Caddy y TLS Automático	11
3.3. Blueprint de App Nativa Android	12
3.3.1. Stack tecnológico recomendado	12
3.3.2. Secuencia de voz en la app Android	12
3.3.3. Roadmap mobile	13
4. Observabilidad, Auto-Curación y Runbook	14
4.1. Stack de Observabilidad	14
4.1.1. Configuración de scrape en Prometheus	14
4.1.2. Reglas de alerta configuradas	15

4.2. Loop de Auto-Curación	15
4.3. Runbook Operativo	16
4.3.1. Despliegue manual de emergencia	16
4.3.2. Rollback manual	17
4.3.3. Rotación de claves	17
4.3.4. Limpieza de imágenes Docker	17
4.3.5. Verificación rápida de salud del stack	17
4.3.6. Alertas y escalación	18

Índice de cuadros

1.1. KPIs del Hermes Stack	1
1.2. Redes Docker del Hermes Stack	2
1.3. Mapa de puertos del stack	3
2.1. Desglose de latencias en el pipeline de voz	5
2.2. Variables de entorno requeridas por LiteLLM	7
2.3. Stack tecnológico del sitio web	8
3.1. GitHub Actions Secrets (Settings → Secrets and variables → Actions)	11
3.2. Variables en /root/.env del VPS (nunca salen del servidor)	11
3.3. Stack tecnológico de la app Android	12
4.1. Capas de auto-curación y tiempos de respuesta	16
4.2. Matriz de alertas y acciones	18

Capítulo 1

Arquitectura General del Sistema

1.1 Resumen Ejecutivo

El **Hermes Stack** es una plataforma de agente conversacional IA completamente autohospedada, desplegada en un único VPS y accesible públicamente a través de subdominios HTTPS con certificados renovados automáticamente.

Propuesta de valor clave

Soberanía total sobre los datos · Coste mensual fijo predecible · Latencia end-to-end de voz menor a 500 ms · Zero-downtime deployment automatizado desde GitHub.

1.1.1 Métricas objetivo

Cuadro 1.1: KPIs del Hermes Stack

Métrica	Descripción	Objetivo
Latencia E2E voz	Audio entrada → Audio salida	< 500 ms
Uptime mensual	Disponibilidad del servicio Hermes	≥ 99.5 %
TTFB LLM	Primer token desde LiteLLM	< 150 ms
Health check	Ciclo de auto-curación Autoheal	30 s
Deploy time	Push a main → servicio actualizado	< 3 min

1.2 Diagrama General del Sistema

El diagrama siguiente muestra los tres planos del sistema: el plano de *usuario* (clientes externos), el plano de *control* (Caddy + GitHub Actions) y el plano de *datos* (servicios internos).

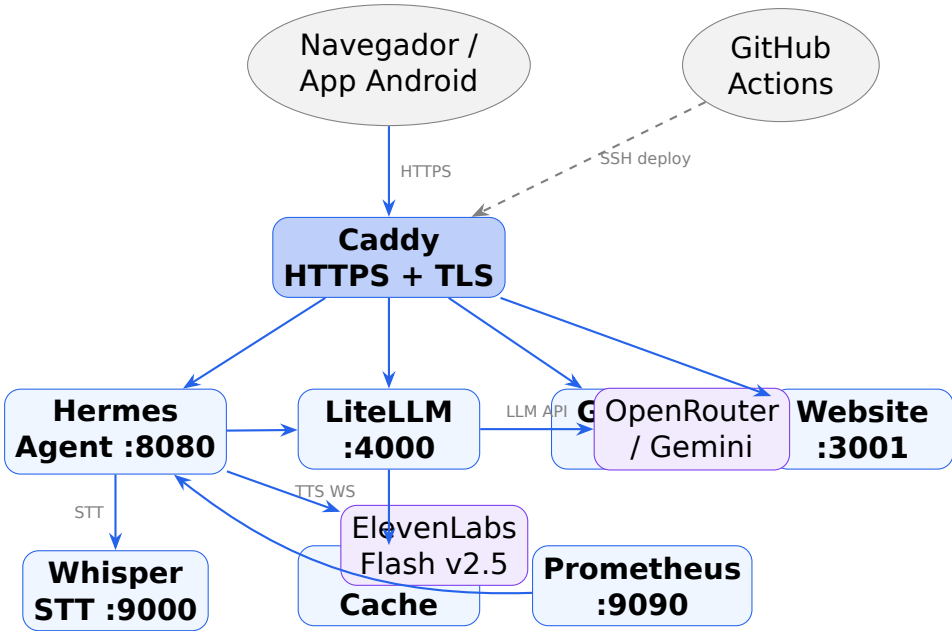


Figura 1.1: Arquitectura general del Hermes Stack

1.3 Principios Arquitectónicos

- 1. **Soberanía de datos:** Todos los datos de usuario transitan por el VPS propio. Sólo salen peticiones a APIs externas (LLM, TTS).
- 2. **Red en capas:** Dos redes Docker aisladas (backend, monitoring). Ningún contenedor de datos está expuesto al host.
- 3. **Secretos en fichero .env:** El servidor jamás expone claves en variables de entorno del sistema operativo ni en el repositorio.
- 4. **Inmutabilidad de contenedores:** Todos los servicios tienen restart: unless-stopped y healthchecks declarativos.
- 5. **Auto-curación:** Autoheal monitoriza cada 30 s y reinicia cualquier contenedor con healthcheck fallido.

1.4 Topología de Red Docker

El stack utiliza dos redes bridge internas:

Cuadro 1.2: Redes Docker del Hermes Stack		
Red	Driver	Miembros
backend	bridge	litellm, redis, whisper-stt, hermes, website, auto-heal, filebrowser
monitoring	bridge	prometheus, grafana, cadvisor, node-exporter, hermes

Aislamiento de red

Los contenedores de *monitoring* no pueden iniciar conexiones hacia los contenedores de *backend* (salvo Hermes y LiteLLM que pertenecen a ambas redes para exponer métricas a Prometheus).

1.4.1 Puertos expuestos al host

Todos los puertos están ligados a 127.0.0.1 (loopback). Caddy actúa como único punto de entrada público en los puertos 80/443.

Cuadro 1.3: Mapa de puertos del stack

Servicio	Puerto host	Subdominio público	Proto
hermes-agent	127.0.0.1:8080	hermes.el80.space	HTTP/WS
litellm-router	127.0.0.1:4000	litellm.el80.space	HTTP
grafana	127.0.0.1:3000	grafana.el80.space	HTTP
hermes-website	127.0.0.1:3001	docs.el80.space	HTTP
whisper-stt	127.0.0.1:9000	— (interno)	HTTP
prometheus	127.0.0.1:9090	— (interno)	HTTP
filebrowser	127.0.0.1:8095	files.el80.space	HTTP
syncthing (opt.)	127.0.0.1:8384	syncthing.el80.space	HTTP

1.5 Hardening de Seguridad en Contenedores

Cada servicio aplica el principio de mínimo privilegio:

```
1 security_opt:
2   - no-new-privileges:true
3 cap_drop:
4   - ALL
5 cap_add:
6   - CHOWN      # Solo cuando es necesario
7   - SETGID
8   - SETUID
9 tmpfs:
10  - /app/tmp:size=64M,noexec,nosuid
```

Listing 1.1: Configuración de seguridad base (docker-compose.yml)

Las excepciones justificadas son:

- **cAdvisor**: requiere privileged: true para leer cgroups del kernel del host.
- **filebrowser**: corre como user: "0:0" para gestionar archivos de root con permisos completos.

- **autoheal**: monta `docker.sock` en modo read-only para detectar el estado de los contenedores.

Capítulo 2

Pipeline de Voz e Inventario de Servicios

2.1 Pipeline End-to-End de Voz

El objetivo central del Hermes Stack es procesar voz en tiempo real con una latencia total menor a 500 ms. El pipeline consta de cinco etapas secuenciales:

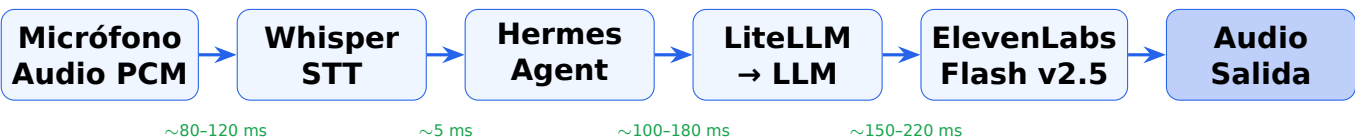


Figura 2.1: Pipeline de voz end-to-end (latencia total objetivo: <500 ms)

2.1.1 Latencias por etapa

Cuadro 2.1: Desglose de latencias en el pipeline de voz

Etapas	Servicio	Descripción	Latencia
1. STT	Whisper (base)	Transcripción audio → texto	80-120 ms
2. Routing	Hermes Agent	Parsing + selección de skill	3-8 ms
3. LLM	LiteLLM	Primer token (TTFB) desde OpenRouter	100-180 ms
4. TTS	ElevenLabs WS	Streaming Flash v2.5 WebSocket	150-220 ms
5. Red/Buffer	VPS → cliente	Transmisión y buffering de audio	10-30 ms
Total E2E			343-558 ms

2.1.2 Optimizaciones de latencia

- **WebSocket persistente con ElevenLabs:** Se reutiliza la conexión WS entre peticiones para eliminar el handshake TLS (~50 ms de ahorro por turno).
- **Streaming de tokens LLM:** LiteLLM reenvía tokens en cuanto llegan; Hermes inicia la llamada TTS con el primer fragmento coherente, no al finalizar la respuesta completa.
- **Modelo Whisper base:** Elegido sobre small o medium para garantizar transcripción en <120 ms en CPU.
- **Redis como caché semántico:** LiteLLM usa Redis para respuestas exactamente repetidas (saludos, comandos frecuentes).

2.1.3 Política de fallback en LiteLLM

```
1 router_settings:
2   routing_strategy: latency-based-routing
3   fallbacks:
4     - model: openrouter/meta-llama/llama-3.1-70b-instruct
5       fallback_models:
6         - gemini/gemini-flash-1.5
7         - openrouter/mistralai/mistral-7b-instruct
8   timeout: 10
9   num_retries: 2
```

Listing 2.1: Fragmento de config/litellm.yaml — fallback

2.2 Inventario Detallado de Servicios

2.2.1 LiteLLM Router (:4000)

LiteLLM actúa como proxy inteligente entre Hermes y los proveedores de LLM externos. Centraliza la autenticación, el enrutamiento basado en latencia y el caché Redis.

```
1 litellm:
2   image: ghcr.io/berriai/litellm:main-latest
3   environment:
4     LITELLM_MASTER_KEY: ${LITELLM_MASTER_KEY}
5     REDIS_HOST: redis
6     OPENROUTER_API_KEY: ${OPENROUTER_API_KEY}
7     GEMINI_API_KEY: ${GEMINI_API_KEY}
8   volumes:
9     - ./config/litellm.yaml:/app/config.yaml:ro
10  ports:
11    - "127.0.0.1:4000:4000"
```

Listing 2.2: docker-compose.yml — servicio litellm

Cuadro 2.2: Variables de entorno requeridas por LiteLLM

Variable	Descripción
LITELLM_MASTER_KEY	Clave maestra para autenticar peticiones al proxy
OPENROUTER_API_KEY	Clave de OpenRouter para acceso a múltiples LLMs
GEMINI_API_KEY	Clave de Google Gemini como fallback primario

2.2.2 Hermes Agent (:8080)

Hermes Agent es el núcleo del sistema. Implementado en Python, expone una API HTTP/WebSocket que orquesta el pipeline completo de voz.

Responsabilidades principales:

- Recibir audio del cliente vía WebSocket
- Llamar a Whisper STT para transcripción
- Enrutar la petición a LiteLLM con el prompt de sistema adecuado
- Transmitir la respuesta de texto a ElevenLabs Flash v2.5
- Hacer streaming del audio de vuelta al cliente
- Exponer métricas Prometheus en /metrics

```
1 async def voice_pipeline(ws: WebSocket):
2     audio_bytes = await ws.recv()
3     transcript = await whisper.transcribe(audio_bytes)
4     llm_stream = litellm.stream(transcript)
5     async for chunk in elevenlabs.tts_stream(llm_stream):
6         await ws.send_bytes(chunk)
```

Listing 2.3: Fragmento ilustrativo — pipeline de voz

2.2.3 Whisper STT (:9000)

Whisper STT corre el modelo base de OpenAI Whisper localmente, garantizando privacidad total de audio.

Configuración recomendada

Usar ASR_MODEL: base para latencia <120 ms en CPU de 2 vCPU. Para CPU de 4+ vCPU, small puede usarse con latencia de ~200 ms pero mejor precisión en acentos.

2.2.4 Redis Cache

Redis 7 Alpine corre con capacidades mínimas:

```
1 redis:
2   image: redis:7-alpine
3   cap_drop: [ALL]
4   cap_add: [CHOWN, SETGID, SETUID]
5   tmpfs:
6     - /tmp:size=32M,noexec,nosuid
```

Listing 2.4: Redis hardening

2.2.5 Hermes Website (:3001)

Hermes Website es una SPA React/Vite con backend Express que sirve la interfaz web del proyecto en docs.el80.space.

Cuadro 2.3: Stack tecnológico del sitio web

Capa	Tecnología
Frontend	React 18 + Vite + TypeScript + Tailwind CSS
Backend	Express (Node 22) — proxy y endpoint /api/tree
Build	Docker multi-stage (builder + runner), imagen Alpine
Assets	Framer Motion para animaciones, Heroicons para iconos

2.2.6 File Browser (:8095)

File Browser es una interfaz web para gestión de archivos del servidor, accesible en files.el80.space. Monta /root como raíz del filesystem navegable.

Seguridad — File Browser

File Browser corre como root (user: "0:0") con acceso de escritura total a /root. Debe protegerse con contraseña fuerte en la UI y acceso SSH-only al VPS.

Capítulo 3

CI/CD, Despliegue y App Android

3.1 Pipeline CI/CD — GitHub Actions

El repositorio incluye un workflow completo en `.github/workflows/deploy.yml` que cubre dos etapas: validación de calidad y despliegue zero-downtime al VPS.

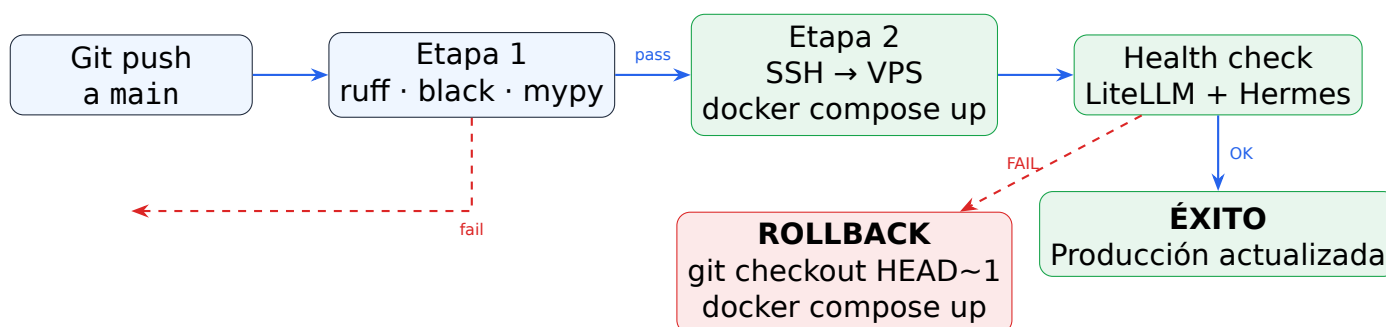


Figura 3.1: Pipeline CI/CD del Hermes Stack

3.1.1 Etapa 1 — Lint & Validation

Se ejecuta en todo PR y push. Nunca accede al servidor de producción.

```
1 lint-and-test:
2   runs-on: ubuntu-24.04
3   steps:
4     - uses: actions/checkout@v4
5     - uses: actions/setup-python@v5
6       with:
7         python-version: "3.12"
8         cache: pip
9     - run: pip install ruff black mypy aiohttp websockets tenacity
10    prometheus_client
11    - run: ruff check ./hermes
```

```

11 - run: black --check ./hermes
12 - run: mypy ./hermes --ignore-missing-imports

```

Listing 3.1: Jobs de validación en deploy.yml

3.1.2 Etapa 2 — Despliegue al VPS

Solo se ejecuta en push a main y requiere que Etapa 1 pase.

```

1 deploy:
2   needs: [lint-and-test]
3   if: github.ref == 'refs/heads/main' && github.event_name == '
      push'
4   steps:
5     - name: Deploy via SSH
6       run: |
7         ssh $VPS_USER@$VPS_HOST "
8           cd /root
9           git pull origin main
10          docker compose up -d --build
11          sleep 15
12
13          # Health check autenticado LiteLLM
14          (source .env && curl -f \
15            -H 'Authorization: Bearer $LITELLM_MASTER_KEY' \
16            http://127.0.0.1:4000/health) \
17          || (git checkout HEAD~1 && docker compose up -d --build
              && exit 1)
18
19          # Health check Hermes Agent
20          curl -f http://127.0.0.1:8080/health \
21          || (git checkout HEAD~1 && docker compose up -d --build
              && exit 1)
22          "

```

Listing 3.2: Despliegue SSH con rollback automático

3.1.3 Secretos requeridos en GitHub

3.1.4 Watchdog del Docker Daemon

docker-watchdog.sh es un servicio systemd que supervisa el daemon de Docker y los contenedores críticos cada 60 segundos, independientemente de Autoheal:

```

1 for item in "litellm-router:litellm" "hermes-agent:hermes" "
      whisper-stt:whisper-stt"; do
2   container="${item%:*}"
3   service="${item#*:}"
4   status=$(docker inspect --format='{{.State.Status}}' "$container
      " 2>/dev/null \

```

Cuadro 3.1: GitHub Actions Secrets (Settings → Secrets and variables → Actions)

Secret	Valor
VPS_HOST	IP pública del servidor (172.236.x.x)
VPS_USER	Usuario SSH (root)
VPS_SSH_KEY	Clave privada SSH (su pública en /root/.ssh/authorized_keys)
VPS_PORT	Puerto SSH (default 22)

Cuadro 3.2: Variables en /root/.env del VPS (nunca salen del servidor)

Variable	Descripción
LITELLM_MASTER_KEY	Clave maestra del proxy LiteLLM
OPENROUTER_API_KEY	API key de OpenRouter
GEMINI_API_KEY	API key de Google Gemini
ELEVENLABS_API_KEY	API key de ElevenLabs
ELEVENLABS_VOICE_ID	ID de voz seleccionada en ElevenLabs

```
5         || echo "missing")
6     if [ "$status" != "running" ]; then
7         log "Contenedor $container caído. Levantando servicio $service"
8         cd "$STACK_DIR" && docker compose up -d --no-deps "$service"
9     fi
10 done
```

Listing 3.3: Lógica central del watchdog

3.2 Configuración Caddy y TLS Automático

Caddy gestiona los certificados TLS mediante el protocolo ACME (Let's Encrypt) automáticamente. Solo requiere que los puertos 80 y 443 estén abiertos en el firewall.

```
1 {
2     email erikjosehernandez@gmail.com
3 }
4
5 hermes.el80.space { reverse_proxy 127.0.0.1:8080 }
6 litellm.el80.space { reverse_proxy 127.0.0.1:4000 }
7 grafana.el80.space { reverse_proxy 127.0.0.1:3000 }
8 docs.el80.space { reverse_proxy 127.0.0.1:3001 }
9 syncthing.el80.space { reverse_proxy 127.0.0.1:8384 }
10 files.el80.space { reverse_proxy 127.0.0.1:8095 }
```

Listing 3.4: /etc/caddy/Caddyfile completo

```
# Repositorio oficial
curl -sLf 'https://dl.cloudsmith.io/public/caddy/stable/gpg.key' \
| gpg --dearmor -o /usr/share/keyrings/caddy-stable-archive-
keyring.gpg

# Firewall
ufw allow 80/tcp && ufw allow 443/tcp

# Habilitar y arrancar
systemctl enable caddy && systemctl restart caddy
```

Listing 3.5: Instalación de Caddy (setup-caddy.sh)

3.3 Blueprint de App Nativa Android

La hoja de ruta incluye una app Android nativa que se conecta directamente al Hermes Stack como cliente de voz y control.

3.3.1 Stack tecnológico recomendado

Cuadro 3.3: Stack tecnológico de la app Android

Capa	Tecnología
Lenguaje	Kotlin + Coroutines + Flow
UI Framework	Jetpack Compose (Material 3)
Audio	MediaRecorder / AudioRecord → PCM 16 kHz mono
Red	OkHttp WebSocket para streaming de voz
LLM directo	Google AI Studio SDK (Gemini Flash en el device)
Persistencia	Room DB para historial de conversaciones
DI	Hilt

3.3.2 Secuencia de voz en la app Android

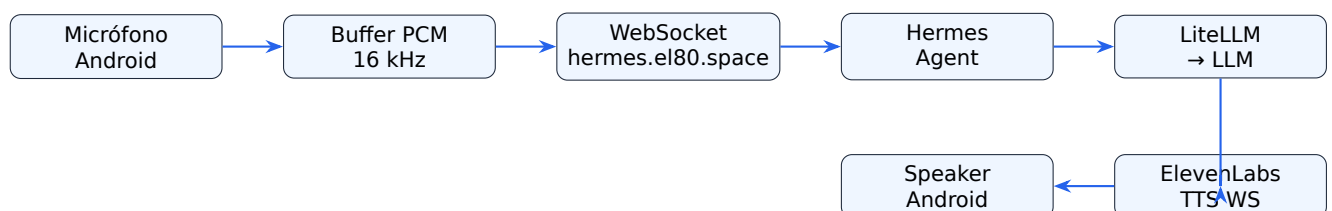


Figura 3.2: Flujo de voz en la app Android

3.3.3 Roadmap mobile

Fase 1: Autenticación con `LITELLM_MASTER_KEY` vía HTTPS

Fase 2: WebSocket de voz bidireccional con Hermes Agent

Fase 3: Dashboard de health (pull de docs.el80.space/api/health)

Fase 4: Llamadas REST directas a LiteLLM para chat sin voz

Fase 5: Integración de Gemini on-device vía Google AI Studio SDK

Capítulo 4

Observabilidad, Auto-Curación y Runbook

4.1 Stack de Observabilidad

El Hermes Stack incluye un stack completo de observabilidad basado en la triada **Métricas** → **Alertas** → **Visualización**:

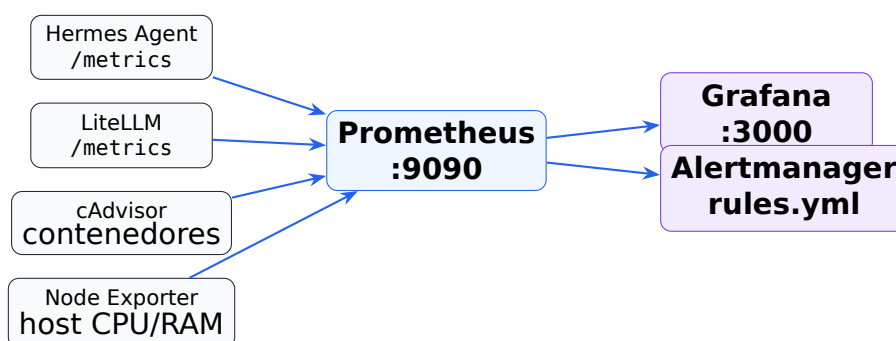


Figura 4.1: Stack de observabilidad — flujo de métricas

4.1.1 Configuración de scrape en Prometheus

```
1 global:
2   scrape_interval:    15s
3   evaluation_interval: 15s
4
5 scrape_configs:
6   - job_name: 'hermes'
7     static_configs:
8       - targets: ['hermes:8080']
9
10  - job_name: 'litellm'
11    static_configs:
12      - targets: ['litellm:4000']
13
```

```
14 - job_name: 'cadvisor'
15   static_configs:
16     - targets: ['cadvisor:8080']
17
18 - job_name: 'node-exporter'
19   static_configs:
20     - targets: ['node-exporter:9100']
```

Listing 4.1: config/prometheus.yml (fragmento)

4.1.2 Reglas de alerta configuradas

```
1 groups:
2   - name: hermes_stack
3     rules:
4       - alert: HermesDown
5         expr: up{job="hermes"} == 0
6         for: 2m
7         labels:
8           severity: critical
9         annotations:
10          summary: "Hermes Agent caído"
11
12       - alert: LiteLLMDown
13         expr: up{job="litellm"} == 0
14         for: 2m
15         labels:
16           severity: critical
17
18       - alert: HighMemoryUsage
19         expr: |
20           container_memory_usage_bytes{name=~"hermes.*|litellm.*"}
21           / container_spec_memory_limit_bytes > 0.85
22         for: 5m
23         labels:
24           severity: warning
25
26       - alert: HighCPULoad
27         expr: 100 - (avg by(instance)(rate(node_cpu_seconds_total
28           {mode="idle"}[5m])) * 100) > 85
29         for: 10m
30         labels:
31           severity: warning
```

Listing 4.2: config/alerts.yml — reglas críticas

4.2 Loop de Auto-Curación

El sistema implementa tres capas de auto-recuperación:

Cuadro 4.1: Capas de auto-curación y tiempos de respuesta

Capa	Herramienta	Acción	Tiempo
1 — Healthcheck	Docker Engine	Marca container unhealthy tras 3 fallos	90 s
2 — Restart	Autoheal	Reinicia container unhealthy	30-120 s
3 — Daemon	docker-watchdog	Verifica y reinicia el daemon Docker	60 s
4 — CI/CD	GitHub Actions	Rollback a commit anterior si deploy falla	3 min

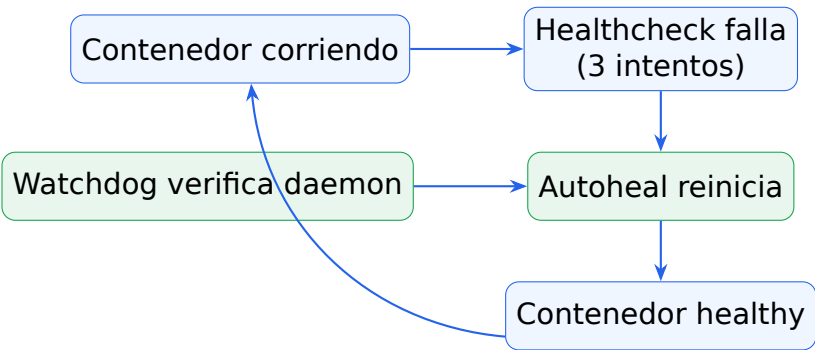


Figura 4.2: Loop de auto-curación del Hermes Stack

4.3 Runbook Operativo

4.3.1 Despliegue manual de emergencia

```
1 cd /root
2
3 # Actualizar codigo
4 git pull origin main
5
6 # Reconstruir y levantar
7 docker compose up -d --build
8
9 # Verificar salud
10 source .env
11 curl -f -H "Authorization: Bearer $LITELLM_MASTER_KEY" \
12     http://127.0.0.1:4000/health
13 curl -f http://127.0.0.1:8080/health
```

Listing 4.3: Despliegue manual desde el VPS

4.3.2 Rollback manual

```
1 cd /root
2 git log --oneline -5          # Identificar el commit objetivo
3 git checkout HEAD~1          # O un hash específico
4 docker compose up -d --build # Reconstruir con versión anterior
5
6 # Verificar y confirmar
7 curl -f http://127.0.0.1:8080/health && echo "Rollback exitoso"
```

Listing 4.4: Rollback al commit anterior

4.3.3 Rotación de claves

Rotación de `LITELLM_MASTER_KEY`:

```
1 # 1. Generar nueva clave
2 NEW_KEY=$(openssl rand -hex 32)
3
4 # 2. Actualizar /root/.env
5 sed -i "s/^LITELLM_MASTER_KEY=.*LITELLM_MASTER_KEY=$NEW_KEY/" /
   root/.env
6
7 # 3. Actualizar el secret en GitHub Actions (via gh CLI o UI)
8 gh secret set LITELLM_MASTER_KEY --body "$NEW_KEY" -R
   erikjosehrndz/hermes-stack
9
10 # 4. Reiniciar LiteLLM para aplicar nueva clave
11 docker compose restart litellm
```

4.3.4 Limpieza de imágenes Docker

Ejecutar mensualmente para liberar espacio en disco:

```
1 # Imágenes sin uso
2 docker image prune -a --filter "until=720h" -f
3
4 # Volúmenes huérfanos (precaución: revisar antes)
5 docker volume ls -qf dangling=true
6 docker volume prune -f
7
8 # Ver espacio recuperado
9 docker system df
```

Listing 4.5: Limpieza de imágenes y volúmenes huérfanos

4.3.5 Verificación rápida de salud del stack

```
1 #!/bin/bash
2 echo "=== Estado de contenedores ==="
3 docker compose ps
4
5 echo "=== Health checks directos ==="
6 curl -sf http://127.0.0.1:8080/health && echo "Hermes: OK" ||
   echo "Hermes: FAIL"
7 curl -sf http://127.0.0.1:9000/docs && echo "Whisper: OK" ||
   echo "Whisper: FAIL"
8 curl -sf http://127.0.0.1:9090/-/ready && echo "Prometheus: OK"
   || echo "Prometheus: FAIL"
9 curl -sf http://127.0.0.1:3000/api/health && echo "Grafana: OK"
   || echo "Grafana: FAIL"
10
11 echo "=== Uso de recursos ==="
12 docker stats --no-stream --format \
13     "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}"
```

Listing 4.6: Health check completo del stack

4.3.6 Alertas y escalación

Cuadro 4.2: Matriz de alertas y acciones

Alerta	Severidad	Acción inmediata
HermesDown	Critical	Verificar logs: docker logs hermes-agent → reiniciar
LiteLLMDown	Critical	Verificar config litellm.yaml y API keys en .env
HighMemoryUsage	Warning	Revisar docker stats → considerar límite de memoria
HighCPULoad	Warning	Identificar proceso con top → escalar VPS si persiste
WhisperDown	Warning	Reiniciar: docker compose restart whisper-stt

Logs centralizados

Para seguir logs de todos los servicios en tiempo real:
docker compose logs -f --tail=50

Para un servicio específico:
docker compose logs -f hermes --tail=100